CENG-322 Mobile Application Development - Deliverable 3

Team Name: IoT Innovators

Project Name: Smart Farm

Team Number: 8

Student's names and IDs:

- Chau Vi Cuong Nguyen – N01634611
- Hoang Tran – N01613506
- Tuan Anh Pham – N01238459
- Truong Anh Nguyen - N01639800

Table of Contents

Member Info and Participation 2
Project Description 3
GitHub Repo link 3
Login functionality 3
List of Sprint Goals 4
Sprint Dashboard 5
Gantt Chart 6
Daily Stand-ups 7
Design Principles and Design Patterns 9
Design Principles 1: Single Responsibility Principle (SRP) 9
Design Principles 2: Dependency Inversion Principle (DIP) 9
Design Pattern: Model – View – View Model (MVVM) 10
Runtime Permission 11
1. Notification Permission (android.permission.POST_NOTIFICATIONS) 11
2. Network State Permission (android.permission.ACCESS_NETWORK_STATE) 12
Two new main functionalities 12
Main Functionality 1: Real-Time Farm Monitoring Dashboard 12
Main Functionality 2: Device Control Panel 13
Customer Feedback in the DB 13

Member Info and Participation

Name	Student ID	GitHub ID	Signature	Effort
Chau Vi Cuong Nguyen	N01634611	ChauViCuongNguyen4611	CVCN	100
Hoang Tran	N01613506	HoangTranTran3506	HT	100
Tuan Anh Pham	N01238459	TuanAnhPham8459	TA	100
Truong An Nguyen	N01639800	TruongAnNguyen9800	TAN	100

Project Description

The Smart Farm IoT application aims to create a comprehensive mobile solution for modern farming management. The final vision is to provide farmers with real-time monitoring and control of IoT-enabled farm equipment, including temperature monitors, automated irrigation systems, and crop health trackers.

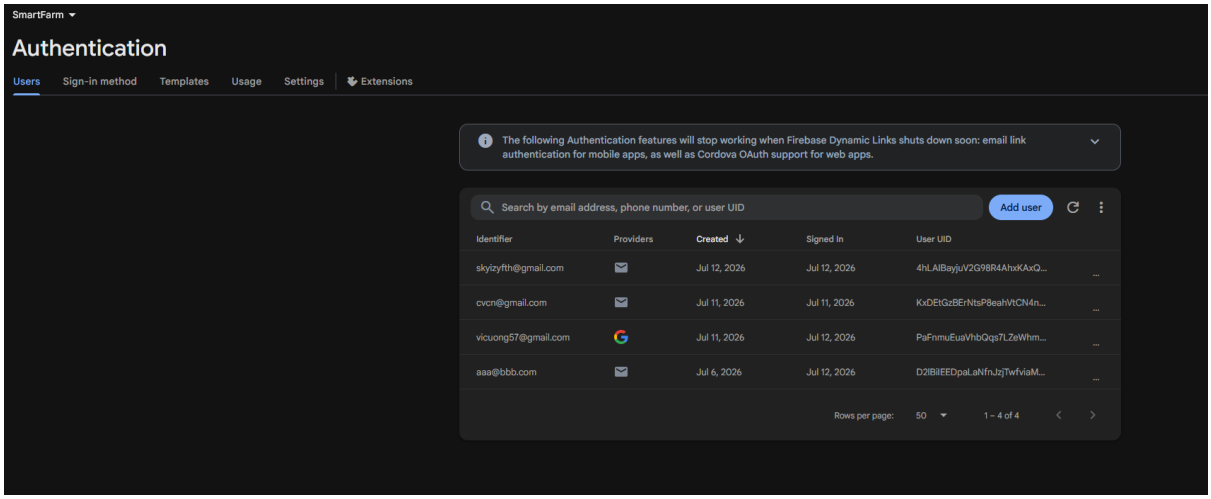
GitHub Repo link

https://github.com/ChauViCuongNguyen4611/SmartFarm

Login functionality

Email: aaa@bbb.com Password: Admin101!

Screenshot showing the Authentication and users logged in using gmail or their email accounts



Describe in detail, the work that has been completed by each team member in this sprint only.

Name	Sprint 3 Work Completed
Chau Vi Cuong Nguyen	Implemented Firebase Authentication, Gmail Sign –In function Implemented function to Registration activity

Hoang Tran	Design feedback, login, sign up layout Implemented Alert Screen Update bottom navigation
Tuan Anh Pham	Implemented Setting, Main, Control Panel, Analytics & Graph Screen Implemented Runtime Permission
Truong An Nguyen	Finished toolbar menu Feedback DB Storage

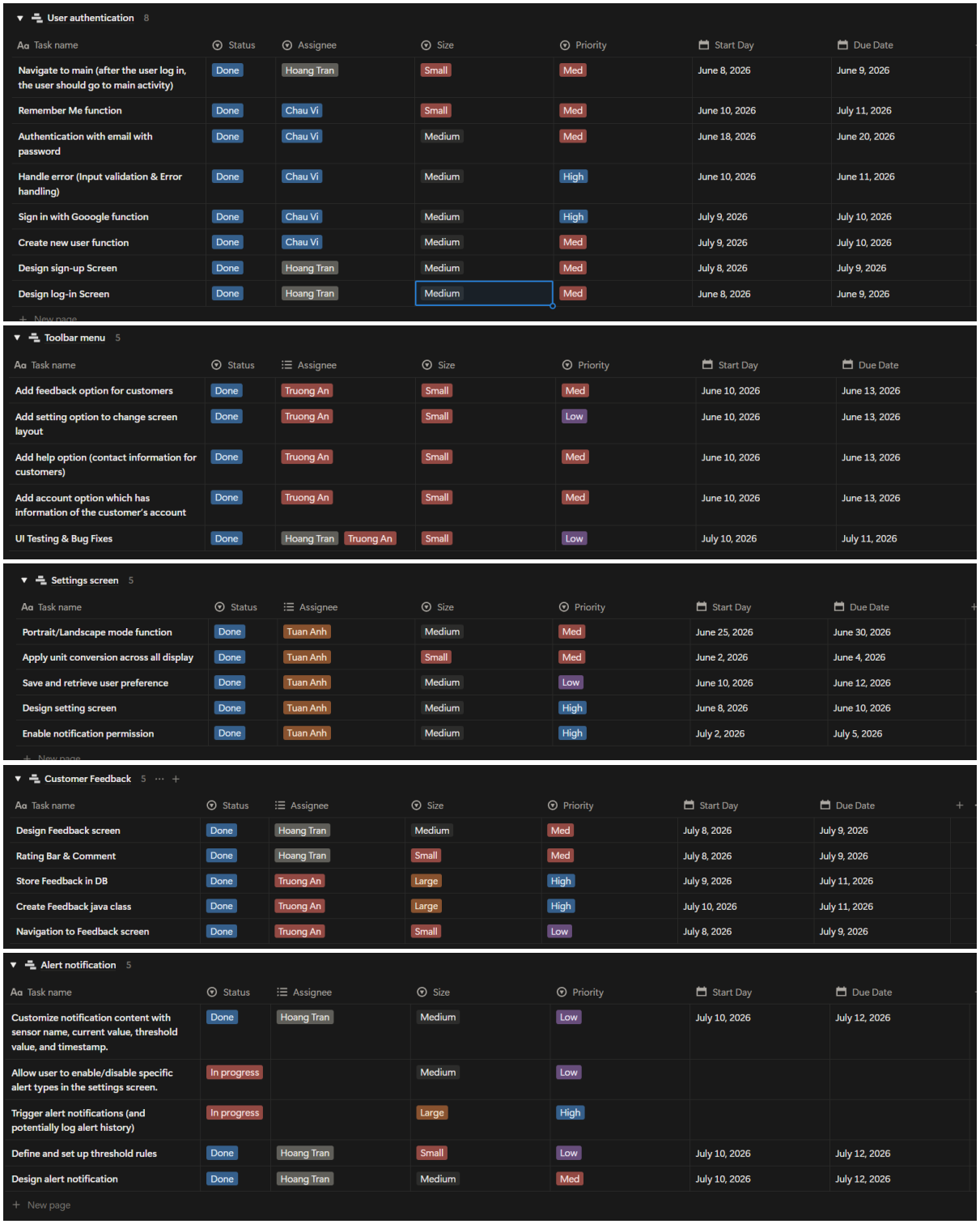
Number of commits by each member for this sprint only.

Name	Number of commits
Chau Vi Cuong Nguyen	15
Hoang Tran	11
Tuan Anh Pham	10
Truong An Nguyen	12

List of Sprint Goals

1. Complete Login + Registration with Firebase Authentication
2. Implement Gmail Sign-In
3. Add “Remember Me” functionality
4. Complete Feedback Screen with DB Storage
5. Update toolbar menu/ bottom navigation/ navigation drawer
6. Finish splash screen/ setting screen and user authentication

Sprint Dashboard



User authentication 8

Task name	Status	Assignee	Size	Priority	Start Day	Due Date
Navigate to main (after the user log in, the user should go to main activity)	Done	Hoang Tran	Small	Med	June 8, 2026	June 9, 2026
Remember Me function	Done	Chau Vi	Small	Med	June 10, 2026	July 11, 2026
Authentication with email with password	Done	Chau Vi	Medium	Med	June 18, 2026	June 20, 2026
Handle error (Input validation & Error handling)	Done	Chau Vi	Medium	High	June 10, 2026	June 11, 2026
Sign in with Gooogle function	Done	Chau Vi	Medium	High	July 9, 2026	July 10, 2026
Create new user function	Done	Chau Vi	Medium	Med	July 9, 2026	July 10, 2026
Design sign-up Screen	Done	Hoang Tran	Medium	Med	July 8, 2026	July 9, 2026
Design log-in Screen	Done	Hoang Tran	Medium	Med	June 8, 2026	June 9, 2026

+ New page

Toolbar menu 5

Task name	Status	Assignee	Size	Priority	Start Day	Due Date
Add feedback option for customers	Done	Truong An	Small	Med	June 10, 2026	June 13, 2026
Add setting option to change screen layout	Done	Truong An	Small	Low	June 10, 2026	June 13, 2026
Add help option (contact information for customers)	Done	Truong An	Small	Med	June 10, 2026	June 13, 2026
Add account option which has information of the customer's account	Done	Truong An	Small	Med	June 10, 2026	June 13, 2026
UI Testing & Bug Fixes	Done	Hoang Tran, Truong An	Small	Low	July 10, 2026	July 11, 2026

Settings screen 5

Task name	Status	Assignee	Size	Priority	Start Day	Due Date
Portrait/Landscape mode function	Done	Tuan Anh	Medium	Med	June 25, 2026	June 30, 2026
Apply unit conversion across all display	Done	Tuan Anh	Small	Med	June 2, 2026	June 4, 2026
Save and retrieve user preference	Done	Tuan Anh	Medium	Low	June 10, 2026	June 12, 2026
Design setting screen	Done	Tuan Anh	Medium	High	June 8, 2026	June 10, 2026
Enable notification permission	Done	Tuan Anh	Medium	High	July 2, 2026	July 5, 2026

+ New page

Customer Feedback 5

Task name	Status	Assignee	Size	Priority	Start Day	Due Date
Design Feedback screen	Done	Hoang Tran	Medium	Med	July 8, 2026	July 9, 2026
Rating Bar & Comment	Done	Hoang Tran	Small	Med	July 8, 2026	July 9, 2026
Store Feedback in DB	Done	Truong An	Large	High	July 9, 2026	July 11, 2026
Create Feedback java class	Done	Truong An	Large	High	July 10, 2026	July 11, 2026
Navigation to Feedback screen	Done	Truong An	Small	Low	July 8, 2026	July 9, 2026

Alert notification 5

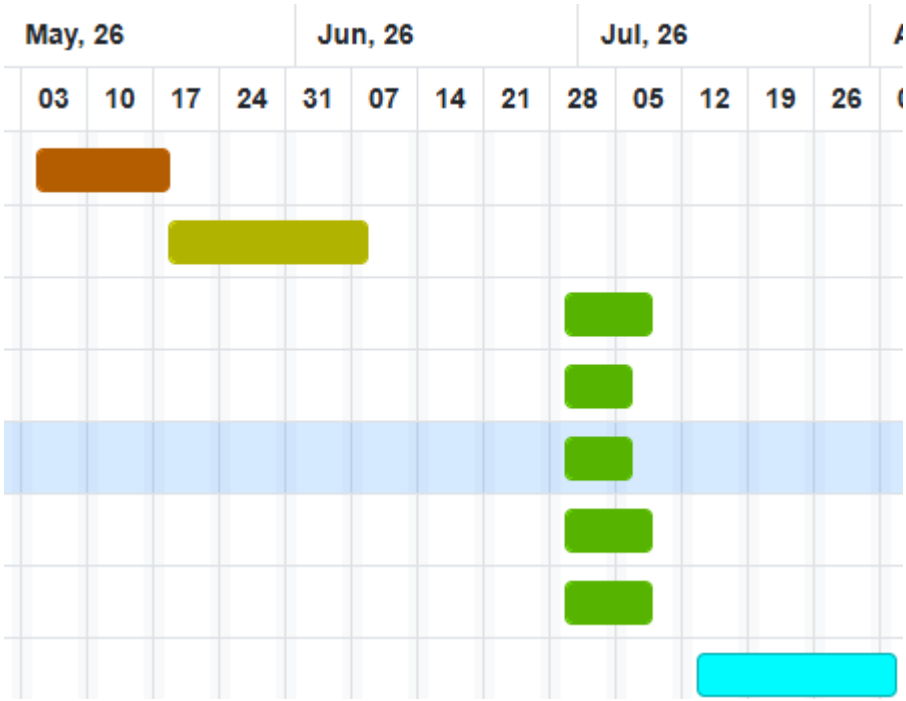
Task name	Status	Assignee	Size	Priority	Start Day	Due Date
Customize notification content with sensor name, current value, threshold value, and timestamp.	Done	Hoang Tran	Medium	Low	July 10, 2026	July 12, 2026
Allow user to enable/disable specific alert types in the settings screen.	In progress		Medium	Low		
Trigger alert notifications (and potentially log alert history)	In progress		Large	High		
Define and set up threshold rules	Done	Hoang Tran	Small	Low	July 10, 2026	July 12, 2026
Design alert notification	Done	Hoang Tran	Medium	Med	July 10, 2026	July 12, 2026

+ New page

Gantt Chart

+ Add | Expand all | Collapse all | Zoom in | Zoom out | Zoom to fit | Search

Name	Start Date	End Date	Duration	Progress %	Dependency	Resources
Deliverable 1	May 04, 2026	May 18, 2026	11 days	100		Everyone
Deliverable 2	May 18, 2026	Jun 08, 2026	16 days	100		Everyone
Deliverable 3	Jun 29, 2026	Jul 08, 2026	8 days	100		Everyone
Firebase Authentication + Gmail Sign-In	Jun 29, 2026	Jul 06, 2026	6 days	100		Chau VI Cuong Nguyen
Feedback, Login, Sign-Up Layout + Alert Scr…	Jun 29, 2026	Jul 06, 2026	6 days	100		Hoang Tran
Settings, Main, Control Panel, Analytics Scre…	Jun 29, 2026	Jul 08, 2026	8 days	100		Tuan Anh Pham
Toolbar Menu + Feedback DB Storage	Jun 29, 2026	Jul 08, 2026	8 days	100		Truong An Nguyen
Deliverable 4	Jul 13, 2026	Aug 03, 2026	16 days	0		Everyone



Daily Stand-ups


Daily Standup
Sprint 3
July 8th, 2026
Timer
Chau Vi Cuong Nguyen
Done
- connect firebase to the app
- finish login using email and password
Do
- login using google account
- check valid email and password
Blockers
No blocker
Hoang Tran
Done
- change the layout for logging activity
- update hardcode into string
Do
- add feedback layout
Blockers
Image's resolution not working as expect
Truong An
Done
- change the toolbar menu item and function
Do
- continue working with toolbar menu item
Blockers
Working with branch in github
Tuan Anh Pham
Done
-Adjusted the UI to improve alignment, spacing, and overall consistency.
-Fixed the layout of the dashboard, control panel, analytics, and graph pages
Do
- Test the responsiveness across different screen sizes
- Refining the layouts
Blockers
No Blockers
Daily Standup
Sprint 3
July 10th, 2026
Chau Vi Cuong Nguyen
Done
- Rewrite/Adding function Input Checker logic
Do
- login using google account
- Signup using email and password
Blockers
No Blockers
Hoang Tran
Done
- finish design for sign up and feedback layout
Do
- Design Alert fragment layout
Blockers
No Blockers
Truong An
Done
- Finish function for Feedback screen
Do
- continue working with feedback database
Blockers
No Blockers
Tuan Anh Pham
Done
-Continued refining the dashboard, control panel, analytics, and graph layouts.
-Tested the UI across different screen sizes and fixed the remaining layout and alignment issues.
Do
-Start implementing additional UI improvement
-Verify pages function correctly
-Fix bugs during testing
Blockers
No Blockers
Daily Standup
Sprint 3
July 12th, 2026
Chau Vi Cuong Nguyen
Done
- Finish user authentication (using email pwd or google account)
Do
- Continue working on alert system
Blockers
No Blockers
Hoang Tran
Done
- finish design alert frag (no func)
- update bottom nav
Do
- update alert screen
- add notification permission to alert frag
Blockers
trouble with merging the branch
Truong An
Done
- Finish working with feedback database
Do
-Update splash screen
-Update
Blockers
No Blockers
Tuan Anh Pham
Done
- finalize every fragment before submission
Do
- Fixing bugs during testing
- Adding runtime permission to the app
Blockers
No Blockers

Design Principles and Design Patterns

Design Principles 1: Single Responsibility Principle (SRP)

A class should have only one reason to change.

Code evidence:

```java
// LoginActivity.java
public class LoginActivity extends BaseActivity implements LoginController.LoginView {
    private TextInputEditText etEmail, etPassword;
    private MaterialButton btnLogin, btnSignup, btnGoogleSignIn;
    private CheckBox cbRememberMe;
    private LoginController loginController;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_login);
        loginController = new LoginController(this, this);
        loginController.onStart();
        initializeViews();
        setupClickListeners();
    }

    private void setupClickListeners() {
        btnLogin.setOnClickListener(v -> {
            String email = etEmail.getText().toString().trim();
            String password = etPassword.getText().toString().trim();
            boolean rememberMe = cbRememberMe.isChecked();
            loginController.handleLogin(email, password, rememberMe);
        });
    }
}

```

LoginActivity is responsible only for UI interactions, including but not limited to initializing views, setting up click listerners, displaying UI feedback, and navigating. This mean if the UI design changes, only LoginActivity needs modifications while the business logic remains untouched.

Design Principles 2: Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules. Both should depend on abstractions (interfaces), not concrete implementations.

Code Evidence:

```java
// LoginController.java - The high-level module depends on an interface
public class LoginController implements FirebaseAuthManager.AuthCallback {
    private final LoginView view;  // Depends on abstraction (interface)

    public interface LoginView {
        void showLoading(boolean isLoading);
        void showEmailError(String err);
        void showPasswordError(String err);
        void onLoginSuccess();
        void onLoginFailure(String errorMessage);
        void navigateToMain();
    }
}

```

LoginController (high-level module) depends on the LoginView interface (abstraction class), not on the concrete LoginActivity class. This allows the controller to work with any UI implementation without modification.

Design Pattern: Model – View – View Model (MVVM)

Separates the application into three interconnected components: Model (data logic), View (UI presentation), and Controller (mediator between Model and View).

```
1. public class LoginActivity extends BaseActivity implements LoginController.LoginView {
2.     // UI elements
3.     private TextInputEditText etEmail, etPassword;
4.     private MaterialButton btnLogin, btnSignup, btnGoogleSignIn;
5.     private CheckBox cbRememberMe;
6.     private LoginController loginController;
7.
8.     @Override
9.     protected void onCreate(Bundle savedInstanceState) {
10.        // Initialize views and controller
11.        loginController = new LoginController(this, this);
12.        initializeViews();
13.        setupClickListeners();
14.    }
15.
16.    // View methods that update the UI
17.    @Override
18.    public void showEmailError(String err) {
19.        tilEmail.setError(err);
20.    }
21.
22.    @Override
23.    public void onLoginSuccess() {
24.        Toast.makeText(this, R.string.login_success, Toast.LENGTH_SHORT).show();
25.        navigateToMain();
26.    }
27. }
28.
```

View (LoginActivity): Handles all UI rendering and user input. It implements LoginView interface to receive updates from the controller.

```
1. public class LoginController implements FirebaseAuthManager.AuthCallback {
2.     private final FirebaseAuthManager authManager;  // Model
3.     private final LoginView view;                   // View
4.     private final InputChecker inputChecker;        // Model
5.     private final LoginPreferences loginPreferences; // Model
6.
7.     public void handleLogin(String email, String password, boolean rememberMe) {
8.         // Validate input (business logic)
9.         InputChecker.ValidationResult result = inputChecker.validateLogin(email, password);
10.        if (!result.isValid()) {
11.            // Update view with errors
12.            view.showEmailError(result.getEmailError());
13.            view.showPasswordError(result.getPasswordError());
14.            return;
15.        }
16.
17.        // Coordinate authentication (business logic)
18.        view.showLoading(true);
19.        authManager.signInWithEmailAndPassword(email, password);
20.    }
21.
22.    // Callback from FirebaseAuthManager (Model)
23.    @Override
24.    public void onSuccess(FirebaseUser user) {
```

```
        view.showLoading(false);
        view.onLoginSuccess();  // Update view
    }
}

```

Controller/Presenter (LoginController): Acts as the middleman, processing user actions, executing business logic, and updating the view through the interface.

```
// FirebaseAuthManager.java - Data/authentication model
public class FirebaseAuthManager {
    private final FirebaseAuth mAuth;

    public void signInWithEmailAndPassword(String email, String password) {
        mAuth.signInWithEmailAndPassword(email, password)
            .addOnCompleteListener(task -> {
                if (task.isSuccessful()) {
                    callback.onSuccess(mAuth.getCurrentUser());
                } else {
                    callback.onFailure(task.getException().getMessage());
                }
            });
    }
}

// InputChecker.java - Validation model
public class InputChecker {
    public ValidationResult validateLogin(String email, String password) {
        // Business logic for validation
        Map<String, String> errors = new HashMap<>();
        String emailError = validateEmail(email);
        if (emailError != null) errors.put(FIELD_EMAIL, emailError);
        // ... more validation
        return new ValidationResult(errors.isEmpty(), errors);
    }
}

// LoginPreferences.java - Data persistence model
public class LoginPreferences {
    public void setRememberMe(boolean rememberMe) {
        editor.putBoolean(KEY_REMEMBER_ME, rememberMe).apply();
    }
}

```

Model (FirebaseAuthManager, InputChecker, LoginPreferences): Manages data, authentication, validation, and persistence logic.

Runtime Permission

Implemented permission:

1. Notification Permission (`android.permission.POST_NOTIFICATIONS`)

The application requests notification permission to allow the system to display alerts and updates to users. This feature is used to notify users about important farm events, such as sensor warnings, abnormal temperature levels, or device status changes.

The permission is declared in `AndroidManifest.xml`:

```
<uses-permission
android:name="android.permission.POST_NOTIFICATIONS"/>
```

At runtime, the application checks whether notification permission has been granted. If permission is not granted, the application requests access from the user. Once approved, the application can send notifications.

2. Network State Permission (android.permission.ACCESS_NETWORK_STATE)

The application uses network state permission to check the current network connection status. This allows the application to determine whether the device is connected to a network before communicating with external smart farm devices or cloud services.

The permission is declared in `AndroidManifest.xml`:

```
<uses-permission
android:name="android.permission.ACCESS_NETWORK_STATE"/>
```

This permission does not require a runtime request because Android grants it automatically during installation.

Two new main functionalities

Main Functionality 1: Real-Time Farm Monitoring Dashboard

The first implemented feature is the **Farm Monitoring Dashboard**.

This feature allows users to monitor the current condition of the smart farm through a single interface. The dashboard displays:

- ESP32 connection status
- Temperature information
- Soil moisture level
- Light level
- Sensor warnings and alerts

The dashboard provides users with a quick overview of farm conditions and helps identify problems, such as unsafe temperature levels.

(Screenshot: Dashboard screen)

Comment:

The dashboard successfully provides a centralized view of sensor data and device status. The visual indicators and organized layout make it easier for users to monitor the farm environment and quickly respond to abnormal conditions.

Main Functionality 2: Device Control Panel

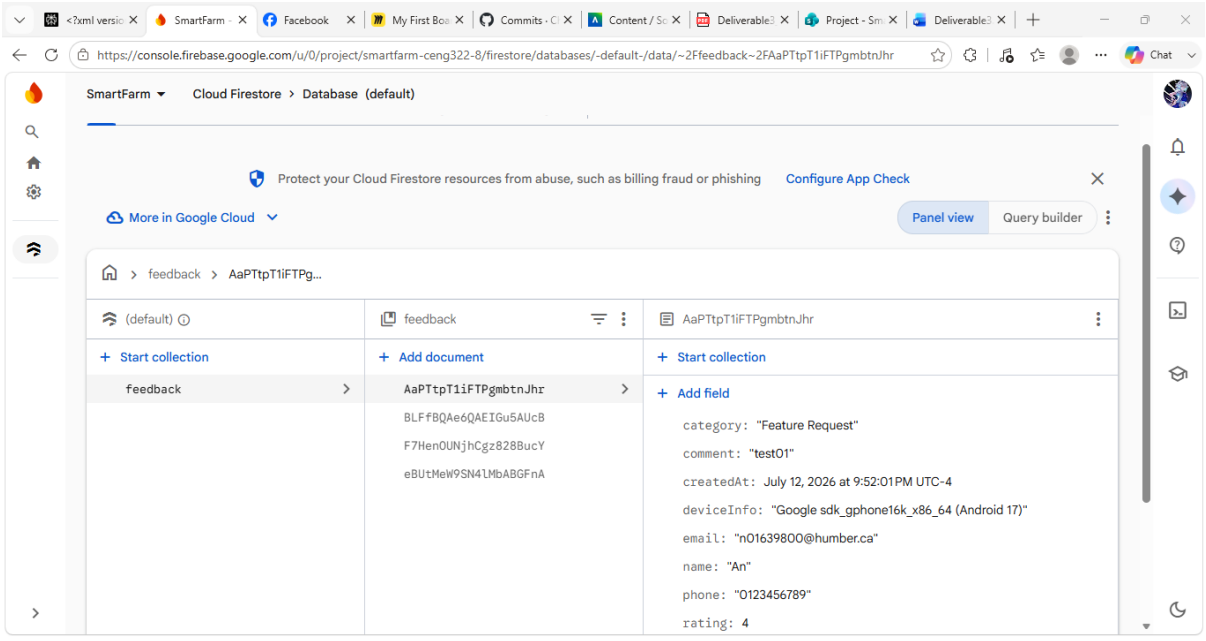
The second implemented feature is the **Device Control Panel**.

This feature allows users to manually control connected farm devices:

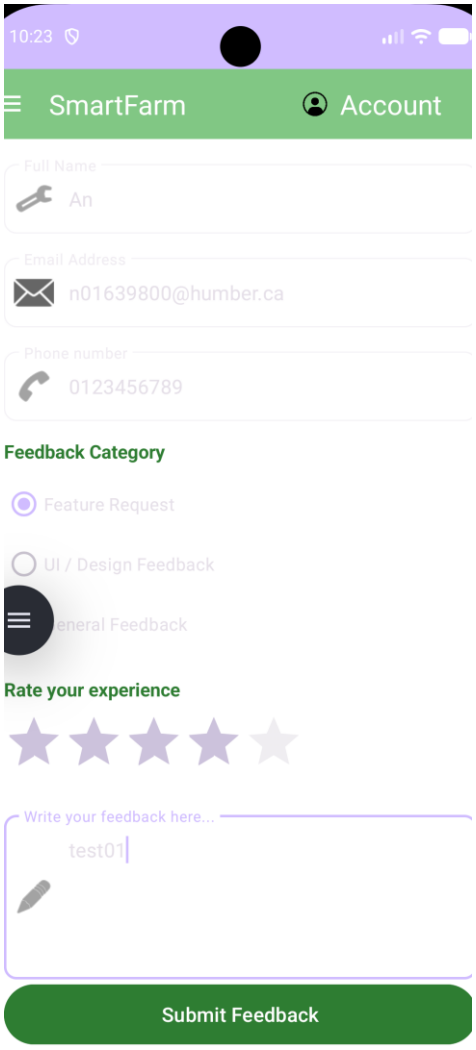
- Irrigation pump
- Cooling fan
- Grow light
- Automatic mode

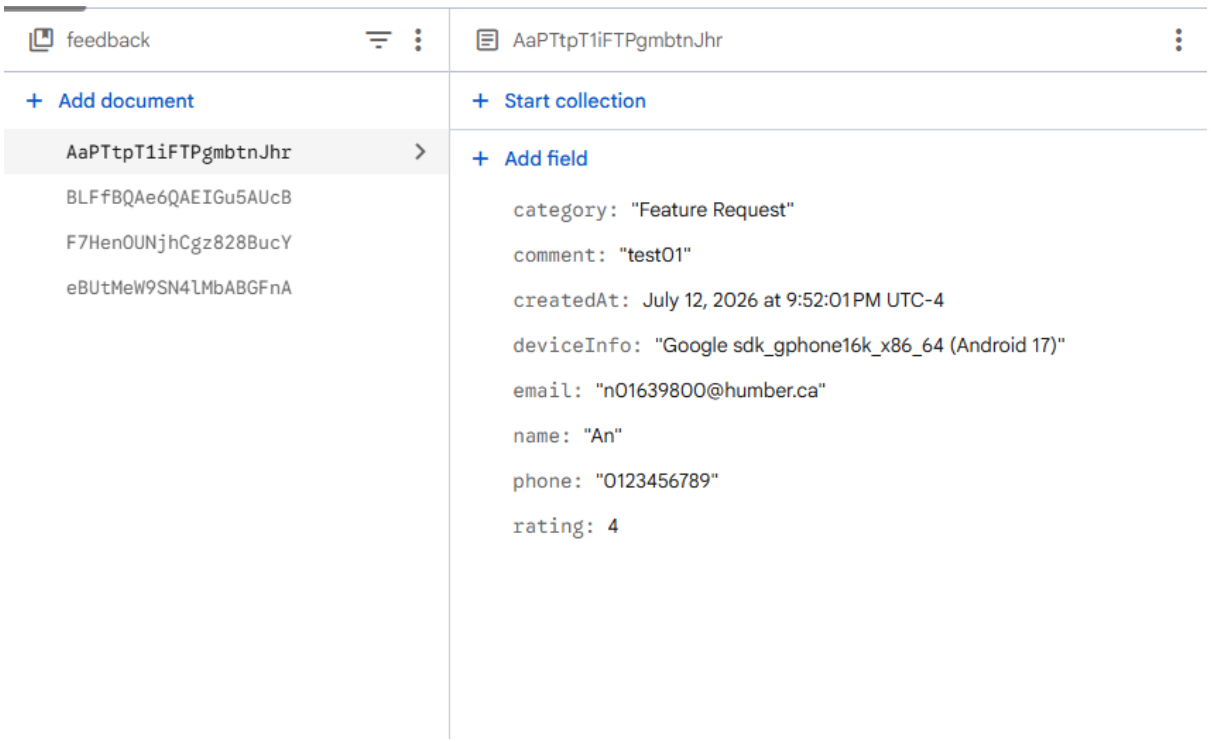
Users can turn devices ON or OFF using switches. The application updates the status display and provides feedback messages when a control action is performed.

Customer Feedback in the DB



Screenshot showing clearly the data transmitted from Feedback screen, and store to DB


10:23
SmartFarm
Account
Full Name
An
Email Address
n01639800@humber.ca
Phone number
0123456789
Feedback Category
Feature Request
UI / Design Feedback
eneral Feedback
Rate your experience
Write your feedback here...
test01
Submit Feedback



How the data from the Customer Feedback Screen stored in the DB?